

Università degli Studi di Parma
Dipartimento di Ingegneria e Architettura
Programmazione di Sistemi Mobili
A.A. 2025/2026



KitchenOS

Sistema intelligente per la gestione della dispensa e pianificazione pasti

Arama Daniel
359908

Indice

1	Introduzione	2
1.1	Scopo dell'applicazione	2
1.2	Soluzione architetturale	2
2	Analisi dei Requisiti	3
2.1	Requisiti Funzionali	3
2.2	Requisiti Non Funzionali	3
3	Struttura del Progetto e Organizzazione del Codice	4
3.1	Struttura del Workspace Android (Client)	4
3.2	Struttura del Backend (Node.js)	4
4	Architettura e la navigazione	5
5	Approfondimento sullo Stack Tecnologico	5
5.1	Android Studio(Frontend)	5
5.1.1	Tecnologie Core	5
5.1.2	Integrazione di Intelligenza Artificiale	6
5.1.3	Persistenza Locale (Room Database)	7
5.1.4	Networking e Serializzazione (Retrofit & Moshi)	8
5.2	Backend e Sicurezza	10
5.2.1	Node.js & Express.js (Architettura a Livelli)	10
5.2.2	Authorization: bcrypt & JWT	10
5.2.3	Database Relazionale Centralizzato (MySQL)	10
6	Guida all'Installazione e all'Uso	13
6.1	Guida all'Installazione	13
6.1.1	Prerequisiti di Sistema	13
6.1.2	Setup del Backend e del Database	13
6.1.3	Setup del Client Android	13
6.2	Flussi dell'Applicazione	14
6.2.1	1. Schermata di Registrazione, Login	14
6.2.2	2. Preferenze Dietetiche (Preferences)	15
6.2.3	3. Dashboard (Home)	15
6.2.4	4. Dispensa (Pantry)	16
6.2.5	5. Pianificatore Settimanale (Piano)	16
6.2.6	6. Ricette Custom	16

1 Introduzione

1.1 Scopo dell'applicazione

Il progetto **KitchenOS** si colloca nel dominio della gestione smart delle risorse domestiche e del *food-tech*. È un'applicazione nata per semplificare la gestione della cucina e ridurre gli sprechi alimentari. Spesso è difficile tenere traccia di tutto il cibo che si ha in casa, rendendo complicato decidere cosa cucinare. L'obiettivo del progetto è creare una vera e propria dispensa digitale che tenga il conto degli ingredienti disponibili in tempo reale. Partendo da questi, l'app suggerisce ricette e pianifica i pasti, rispettando i gusti e le necessità nutrizionali dell'utente.

1.2 Soluzione architetturale

La soluzione implementata è un sistema distribuito composto da un'applicazione client nativa Android e un'infrastruttura backend orientata ai servizi (RESTful). Il sistema orchestra diverse tecnologie:

- **Frontend Mobile:** Sviluppato in Kotlin adottando il framework dichiarativo Jetpack Compose per il rendering reattivo della UI.
- **Persistenza Dati e Caching:** Utilizzo di Room Database per garantire la *Local Consistency* e il funzionamento offline-first, sincronizzato con il cloud.
- **Backend & Identity and Access Management:** Implementato in Node.js/Express.js, gestisce la logica centralizzata, l'autenticazione *stateless* e il routing verso MySQL.
- **Integrazione API di terze parti:** Aggregazione dati tramite micro-servizi esterni (Spoonacular per l'analisi e raccomandazione ricette, OpenFoodFacts per la decodifica dei codici a barre e l'estrazione dei macro-nutrienti).

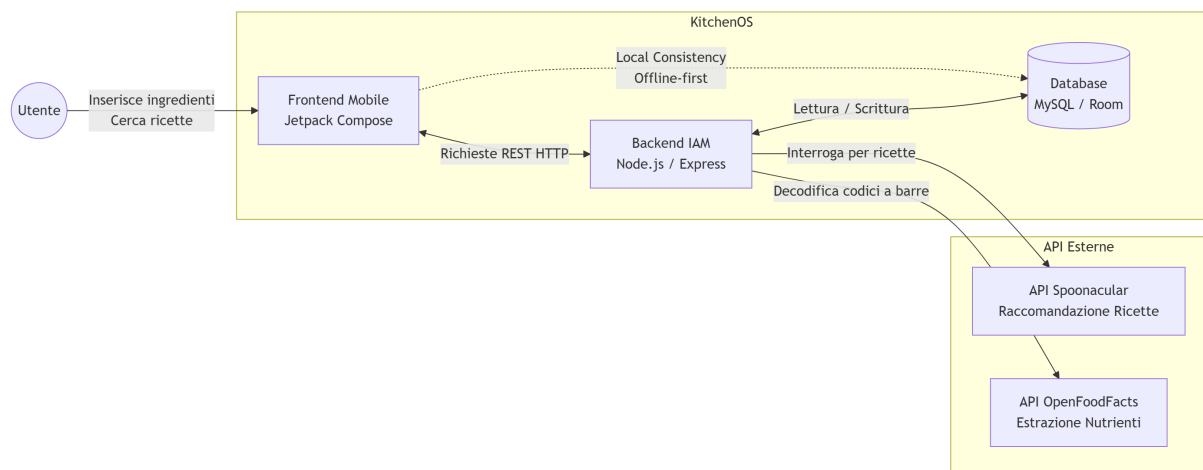


Figura 1: Diagramma architetturale di KitchenOS.

2 Analisi dei Requisiti

2.1 Requisiti Funzionali

I requisiti funzionali definiscono le interazioni dirette tra l'utente e il sistema:

- **Gestione Login/Register:** Il sistema consente la registrazione e il login sicuro degli utenti, mantenendo sessioni prolungate per evitare accessi ripetuti e garantendo la riservatezza dei dati personali.
- **Personalizzazione Dietetica:** Acquisizione e il salvataggio di vincoli dietetici (es. regimi vegano/vegetariano, intolleranze specifiche come glutine o lattosio, budget di tempo per la preparazione, preferenze su specifiche cucine etniche).
- **Inventory Management (Dispensa):** Capacità di inserire prodotti tramite inserimento manuale ,scansione ottica (Barcode reader) , o image recognition . Il sistema deve interpolare il codice EAN con database esterni per auto-compilare i metadati del prodotto (nome, brand, quantità, unità di misura).
- **Meal Planner Algoritmico:** Un modulo dedicato alla generazione dinamica di un piano pasti settimanale. Il calcolo deve contenere: i filtri del profilo utente, le disponibilità attuali nel database locale (Room) e le query al Spoonacular.
- **Dettagli Ricette:** Visualizzazione delle procedure di preparazione, dei tempi di cottura e verifica istantanea della disponibilità degli ingredienti necessari.
- **Cache locale:** Memorizzare ricette e dati utente sul dispositivo per ridurre i tempi

2.2 Requisiti Non Funzionali

I requisiti non funzionali dettano i parametri di qualità e le performance del sistema:

- **Architettura Offline-First:** L'app implementa una logica di *Caching* robusta. I dati essenziali devono risiedere in un database SQLite locale (Room) per garantire tempi di risposta sotto i 200ms nelle transizioni di UI e permettere la consultazione della dispensa anche in assenza di connettività.
- **Sicurezza e Crittografia:** Le password vengono cifrate prima del salvataggio e non transitano mai in chiaro. Lo scambio di dati tra client e server è protetto e validato tramite token JWT.
- **Usabilità:** Interfaccia molto user-friendly , basata su Compose , con navigazione chiara

3 Struttura del Progetto e Organizzazione del Codice

Per garantire la massima manutenibilità e scalabilità, i repository sono stati strutturati seguendo questi principi.

3.1 Struttura del Workspace Android (Client)

Il progetto Android segue una chiara suddivisione logica dei package:

- `com.kitchenos.app.ui`: Contiene tutte le componenti grafiche Jetpack Compose, suddivise per feature.
- `com.kitchenos.app.viewmodel`: Ospita i ViewModel che mantengono lo stato della UI (StateFlow) e orchestrano la logica di business.
- `com.kitchenos.app.data.local`: Contiene l'implementazione del database Room (Entities, DAO, e la classe astratta del DB).
- `com.kitchenos.app.data.model`: Ospita le data class e enums
- `com.kitchenos.app.data.api`: Contiene tutte le chiamate API eseguito durante il processo e le interfacce Retrofit e Moshi.

3.2 Struttura del Backend (Node.js)

L'architettura lato server segue il pattern MVC (Model-View-Controller) riadattato per API REST:

- `/routes`: Definizione degli endpoint RESTful.
- `/env`: Le variabili usati durante la produzione
- `/db.js`: L'inizializzazione e gestione della connessione del database MySQL.
- `/mysql.sql`: Definisce l'intero schema del database relazionale

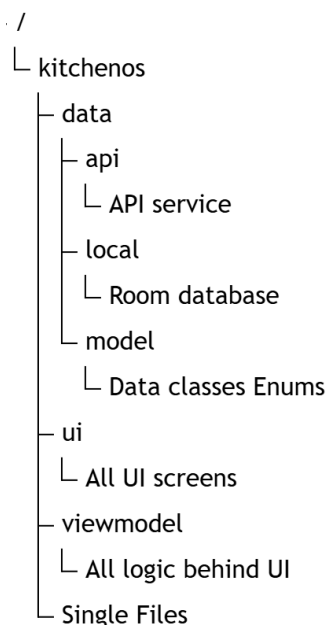


Figura 2: Struttura Front

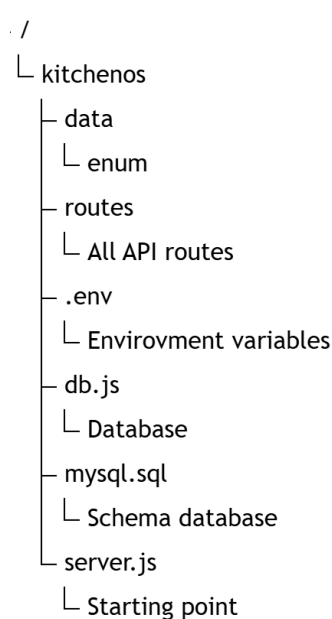


Figura 3: Struttura Back

4 Architettura e la navigazione

Modello Architetturale: MVVM

L'app Android è basata sull'architettura **MVVM (Model-View-ViewModel)**

- **La View (Compose):** È una rappresentazione dello stato. Ascolta uno `StateFlow` dal `ViewModel` e si aggiorna automaticamente quando i dati cambiano.
- **Il ViewModel:** Riceve gli eventi da View, e fa la logica

La navigazione (Compose Navigation)

La transizione tra le schermate è gestita dalla libreria **Navigation Component for Compose**. L'App contiene due gestioni della navigazione all'interno:

- **Routing di Autenticazione:** dedicato solamente alle schermate di Login, Register, Preferences
- **Routing Principale(Navbar):** dedicata alla navigazione interna

Entrambe usano il componente `NavHost` che è nativo di Jetpack Compose, per creare un grafo di navigazione.

```
NavHost(  
    navController = navController,  
    startDestination = startDestination  
) {  
    composable("login") { ... }  
    composable("register") { ... }  
    composable("preferences") { ... }  
    composable("home") { ... }  
}
```

5 Approfondimento sullo Stack Tecnologico

L'architettura software di KitchenOS è suddivisa in due macro-sistemi: un client mobile nativo (Frontend Android) e un server centrale di computazione e persistenza (Backend Node.js con MySQL).

5.1 Android Studio(Frontend)

5.1.1 Tecnologie Core

- **Kotlin & Coroutines (Gestione Asincrona):**
 - *Cosa ho fatto:* Ho implementato l'intera logica ai dati sfruttando i costrutti asincroni nativi di Kotlin, isolando le operazioni pesanti (I/O sul database Room, chiamate di rete con Retrofit, elaborazione di immagini per il computer vision) su thread pool dedicati (`Dispatchers.IO` e `Dispatchers.Default`).

- *Perché l'ho scelto:* L'utilizzo delle Coroutines e dei flussi asincroni (StateFlow, SharedFlow) elimina completamente il rischio di bloccare il thread principale dell'applicazione (Main Thread), cioè bloccare la UI del utente.

- **Jetpack Compose & Navigation Compose:**

- *Cosa ho fatto:* Ho interamente rimosso il paradigma legacy basato sui file di layout XML. Tutta l'interfaccia utente è diventata molto dinamico e reativa.
- *Perché l'ho scelto:* Compose è più facile e capibile. Questo azzerava i bug legati alla sincronizzazione dello stato grafico.

5.1.2 Integrazione di Intelligenza Artificiale

Per ottimizzare la memorizzazione e l'inventario all'interno della dispensa intelligente, sono stati integrati due distinti livelli di visione artificiale on-device:

- **Google ML Kit (Barcode Scanning API):**

- *Cosa ho fatto:* Ho implementato un modulo di scansione ottica in tempo reale integrato con la fotocamera del dispositivo (tramite CameraX). ML Kit intercetta i frame video, rileva la presenza di codici a barre standard (EAN-13) e ne estrae il payload numerico stringa.
- *Perché l'ho scelto:* Fornisce un'accuratezza e una velocità di lettura sub-secondaria senza gravare sulla CPU del dispositivo, gestendo automaticamente problemi di orientamento, distorsione prospettica e scarsa illuminazione. Ed è gratis

- **Custom Machine Learning Model (Classificazione Frutta e Verdura):**

- *Cosa ho fatto:* Poiché i prodotti freschi (ortofrutta) sono sprovvisti di codice a barre, ho addestrato e integrato un modello di Deep Learning personalizzato di Object Classification, il modello è stato addestrato basandosi sull'architettura MobileNetV2 utilizzando il framework PyTorch. Successivamente, per consentirne l'esecuzione in ambiente Android, il modello è stato convertito nel formato .ptl, ottimizzato per l'integrazione tramite PyTorch Mobile. (codice di python con il addestramento insieme col progetto)
- *Perché l'ho scelto:* Consente all'applicazione di identificare visivamente gli ingredienti non tracciabili tramite codice a barre (es. "mela", "pomodoro", "carota"), Il modulo di Image Labeling standard di Google ML Kit non garantisce una classificazione degli alimenti, restituendo etichette generiche ("cibo") che non permettono l'identificazione precisa dei singoli ingredienti!

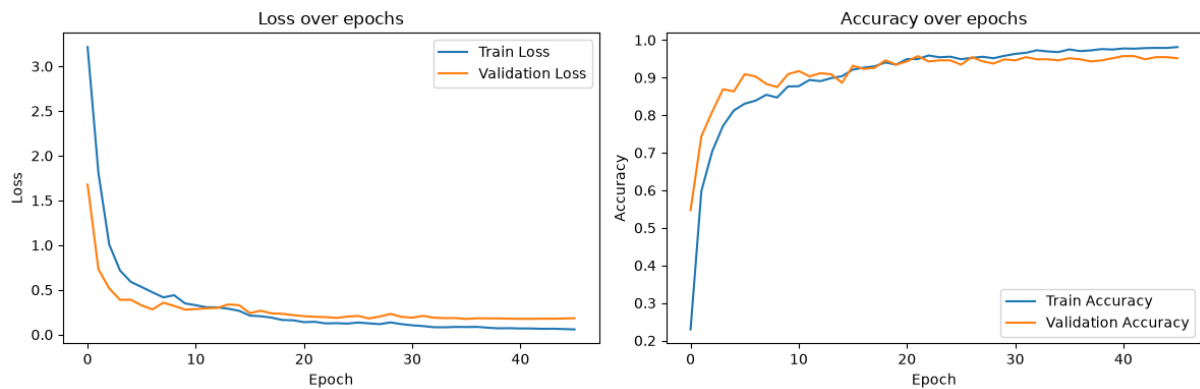


Figura 4: 50 Cicli di addestramento

5.1.3 Persistenza Locale (Room Database)

Per supportare un'architettura Offline-First e garantire la disponibilità dei dati anche in assenza di connessione, lo strato di persistenza locale è stato affidato al Room.

Mappatura delle Tabelle (Entities): Il database locale è strutturato in quattro tabelle principali:

- **PantryEntity:** Gestisce l'inventario reale (ID prodotto, nome, quantità disponibile, unità di misura, URL immagine).
- **PreferencesEntity:** Memorizza localmente il profilo dietetico dell'utente.
- **CachedMealEntity:** Funge da cache per l'indice generale delle ricette disponibili che contiene (id, immagine, titolo, ingredienti, passaggi, cucina e altri).
- **WeeklyPlanEntity:** Associa i giorni della settimana agli ID delle ricette pianificate.
- **CustomMealEntity:** E' una ricetta personalizzata dal utente. Simile come CacheMealEntity

Esempio di Codice: DAO Ogni Tabella del Database , usa gli stessi metodi Principali : ADD, REMOVE, GET, SAVE.
Alcuni DAO hanno anche delle funzioni Flow , quando quella tabella si cambia , fa partire un evento

```
@Dao
interface PreferencesDao{
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun savePreferences(preferences: PreferencesEntity)

    @Query("SELECT * FROM user_preferences WHERE userid = :userId")
    suspend fun getPreferences(userId: String): PreferencesEntity?

    @Query("SELECT * FROM user_preferences WHERE userid = :userId")
    fun getPreferencesFlow(userId: String): Flow<PreferencesEntity?>

    @Query("DELETE FROM user_preferences WHERE userid = :userId")
    suspend fun clearUserPreferences(userId: String)
}
```

5.1.4 Networking e Serializzazione (Retrofit & Moshi)

La comunicazione asincrona verso gli endpoint RESTful del backend è gestita dal framework **Retrofit**, accoppiato a **Moshi** per la gestione del parsing JSON in classi fortemente tipizzate (Data Class).

Orchestrazione delle Chiamate API: L'interfaccia di rete centralizza tutte le interazioni remote necessarie per il funzionamento del sistema applicativo. Le chiamate comprendono i processi di:

- Autenticazione (Scambio di credenziali e ricezione del token JWT).
- Sincronizzazione dello stato della dispensa e delle preferenze utente.
- Richiesta algoritmica di generazione dei piani pasto .

Implementazione dei TypeConverters: SQLite supporta nativamente solo tipi di dati primitivi. Per memorizzare strutture dati complesse (come liste di ingredienti o enumerazioni), ho implementato dei `@TypeConverter` che utilizzano internamente la libreria Moshi per serializzare le strutture dati in stringhe JSON e deserializzarle in oggetti Kotlin.

Esempio di Codice: Definizione dell'ApiService Retrofit

```
interface ApiService {
    @POST("auth/register")
    suspend fun register(
        @Body request: RegisterRequest
    ): Response<AuthResponse>
```

```

@POST("auth/login")
suspend fun login(
    @Body request: LoginRequest
): Response<AuthResponse>

@GET("user/preferences")
suspend fun getUserPreferences(
    @Header("Authorization") token: String
): Response<UserPreferencesRequest>

@POST("user/preferences")
suspend fun savePreferences(
    @Header("Authorization") token: String,
    @Body request: UserPreferencesRequest
): Response<GenericResponse>
}

```

- register : Registra un utente
- login: Logga un utente
- getUserPreferences : Prende gli dati(preferenze) del utente dal server
- savePreferences: Salvare dati utente
- getRecipeByType : Crea un piano pasto , in base al tipo (Colazione, Cena, Pranzo)
- getCurrentPlant : Prende la settimana attuale
- saveCurrentPlan : Salva la settimana attuale generata
- verifyProduct: Manda il barcode e attraverso OpenFoodFacts controlla se esiste, puoi attraverso Spoonacular ricevo l'ingrediente
- getRemoteRecipes: Riceve le ricette custom dal server
- addRemoteRecipe: Aggiunge una ricetta custom
- deleteRemoteRecipe : Cancella una ricetta custom

5.2 Backend e Sicurezza

Il backend è strutturato per fungere da strato computazionale centralizzato, proteggendo l'accesso alle risorse e interfacciando l'applicazione con i database relazionali e le API esterne di terze parti.

5.2.1 Node.js & Express.js (Architettura a Livelli)

- *Cosa ho fatto:* Ho sviluppato un server HTTP RESTful modulare basato su Express.js. Il server espone endpoint protetti ed effettua chiamate server-to-server verso **Spoonacular** e **OpenFoodFacts** per l'arricchimento dei dati.

5.2.2 Authorization: bcrypt & JWT

- **Crittografia Simmetrica con bcrypt:**
 - *Cosa ho fatto:* Durante la registrazione, la password dell'utente non viene memorizzata in chiaro all'interno del database MySQL. Il testo subisce un processo di hashing tramite l'algoritmo bcrypt applicando un fattore di costo (*work factor*) pari a 10.
- **Autenticazione Stateless tramite JSON Web Token (JWT):**
 - *Cosa ho fatto:* Al termine di una procedura di login corretta, il server genera un token JWT contenente l'ID dell'utente e la data di scadenza (configurata a 7 giorni),

5.2.3 Database Relazionale Centralizzato (MySQL)

I dati persistenti globali dell'applicazione sono storicizzati all'interno di un **MySQL**.

Modellazione delle Tabelle e Relazioni: Lo schema logico del database relazionale centrale si articola sulle seguenti entità collegate:

```
CREATE DATABASE kitchenos;
USE kitchenos;

-- Tabella Utenti : Memorizza le credenziali fondamentali del utente
CREATE TABLE Users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  username VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabella Preferenze : Memorizza le preferenze
CREATE TABLE Preferences (
  user_id INT PRIMARY KEY,
  dietary_regime VARCHAR(50),
```

```

    diet_type JSON,
    time_preparation VARCHAR(50),
    allergies JSON,
    cousine JSON,
    FOREIGN KEY (user_id) REFERENCES Users(id) ON DELETE CASCADE
);

```

-- Tabella WeekPlan : Memorizza il piano settimanale

```

CREATE TABLE meal_entity (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    spoonacularId BIGINT,
    image VARCHAR(512),
    title VARCHAR(255),
    readyInMinutes INT,
    servings INT,
    vegetarian BOOLEAN,
    vegan BOOLEAN,
    glutenFree BOOLEAN,
    dairyFree BOOLEAN,
    summary TEXT,
    spoonacularScore DOUBLE,

    extendedIngredients JSON,
    nutrition JSON,
    cuisines JSON,
    dishTypes JSON,
    analyzedInstructions JSON,

    dayOfWeek VARCHAR(50),
    mealType VARCHAR(50),

    fetchedTimestamp BIGINT,
    FOREIGN KEY (user_id) REFERENCES Users(id) ON DELETE CASCADE
);

```

-- Tabella Ricette Custom : Memorizza le ricette custom

```

CREATE TABLE custom_meals (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    userid VARCHAR(255) NOT NULL,
    spoonacularId BIGINT DEFAULT NULL,
    image VARCHAR(1000) DEFAULT NULL,
    title VARCHAR(255) NOT NULL,
    readyInMinutes INT NOT NULL,
    servings INT NOT NULL,
    summary TEXT,
    spoonacularScore DOUBLE DEFAULT NULL,

```

```

extendedIngredients JSON,
nutrition JSON,
cuisines JSON,
dishTypes JSON,
analyzedInstructions JSON
);

```

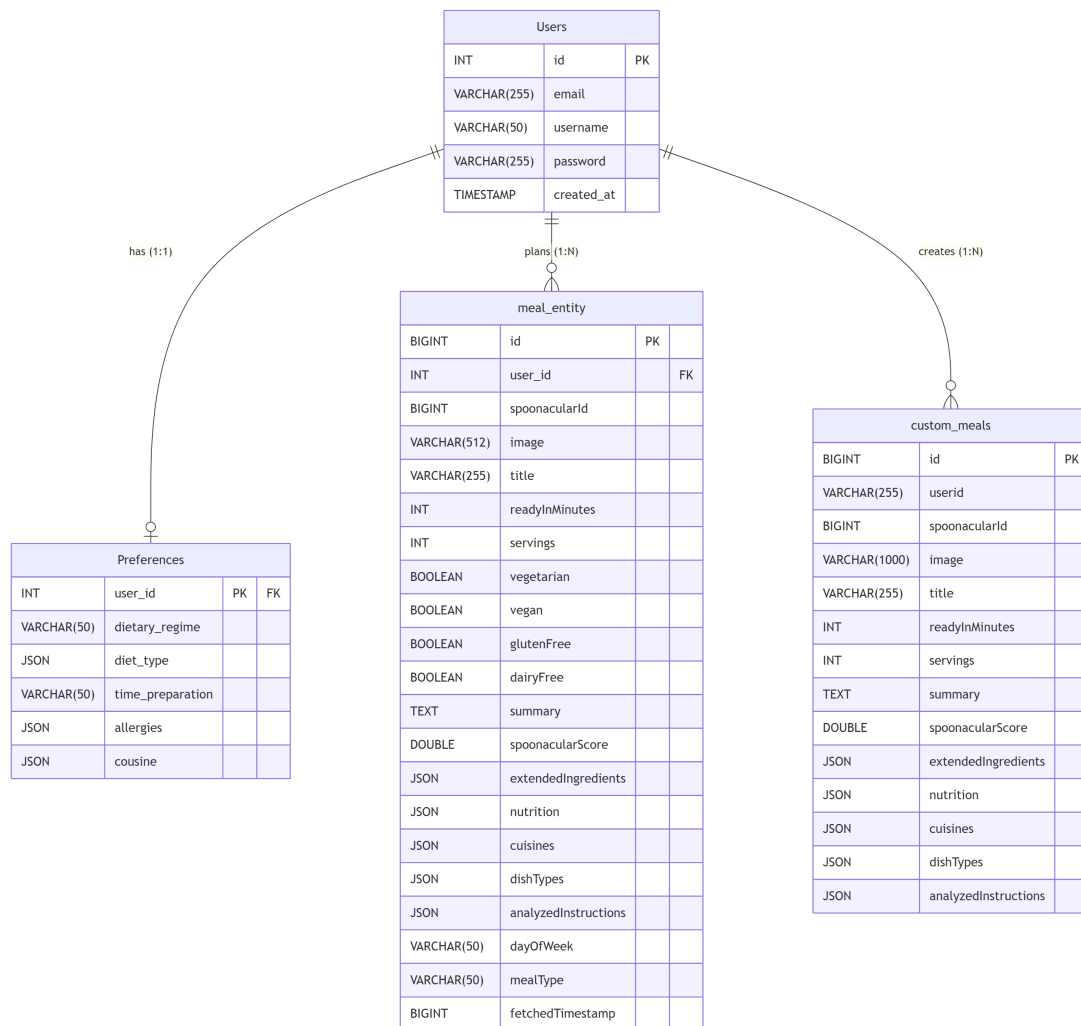


Figura 5: Diagramma Entity-Relationship (ER) dello schema logico del database MySQL.

6 Guida all'Installazione e all'Uso

6.1 Guida all'Installazione

L'avvio del sistema richiede la configurazione di due ambienti separati: il server Node.js e l'applicazione Android.

6.1.1 Prerequisiti di Sistema

- **Ambiente Server:** Node.js (v18+), un'istanza di MySQL Server (v8.0+) attiva.
- **Ambiente Client:** Android Studio, dispositivo fisico o emulatore con API Level 26+.

6.1.2 Setup del Backend e del Database

1. **Inizializzazione Database:** Accedere alla shell di MySQL (o tramite un client come MySQL Workbench) ed eseguire lo script `mysql.sql` fornito nella repository. Questo comando creerà il database `kitchenos` e istanzierà lo schema relazionale completo.
2. **Installazione Dipendenze:** Aprire il terminale nella directory radice del backend ed eseguire il comando `npm install` per scaricare i pacchetti necessari
3. **Configurazione Variabili d'Ambiente:** Creare un file `.env` nella root del server per definire le credenziali di accesso:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=latuapassword
DB_NAME=kitchenos
JWT_SECRET=una_chiave_segreta_molto_complessa
SPOONACULAR_API_KEY=la_tua_chiave_api
```

4. **Avvio del Server:** Avviare il servizio RESTful tramite il comando `npm start`. Il server si porrà in ascolto sulla porta specificata (es. `http://localhost:3000`).

6.1.3 Setup del Client Android

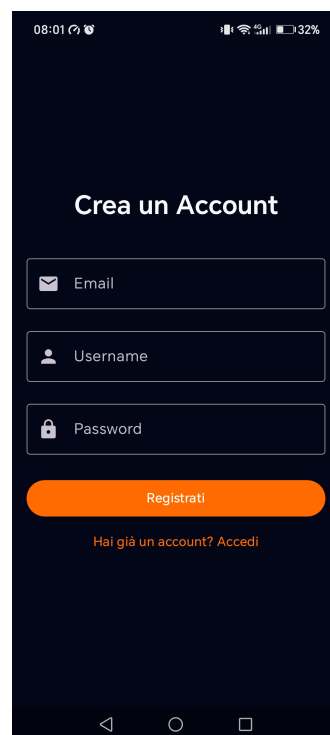
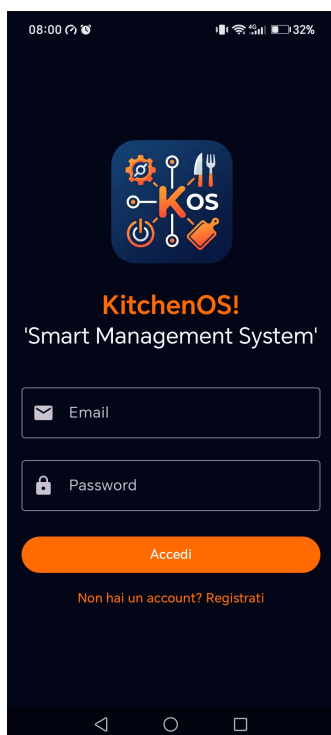
1. Aprire il progetto in Android Studio e attendere la sincronizzazione di Gradle.
2. **Configurazione di Rete:** Poiché l'emulatore Android gira in una macchina virtuale isolata, per comunicare con il backend locale è necessario modificare l'indirizzo IP nel file `RetrofitClient.kt`. Sostituire `localhost` con l'IP della propria macchina sulla rete locale (es. `192.168.1.x`) o con `10.0.2.2` se si utilizza l'emulatore standard di Android.
3. **Compilazione:** Premere il tasto *Run* (Shift + F10) per compilare il pacchetto `.apk` e installarlo sul dispositivo target.

6.2 Flussi dell'Applicazione

6.2.1 1. Schermata di Registrazione, Login

LoginScreen, RegisterScreen , AuthViewModel Rappresenta il punto di ingresso per i utenti. Il modulo richiede l'inserimento di un indirizzo email valido, uno username univoco e una password.

- **Logica applicata:** I dati vengono validati lato client (controllo formato email e lunghezza password) prima di essere inviati all'endpoint `/auth/register` o `/auth/login`.
- In caso di Login , il utente viene reindirizzato all home , pero in caso di Register il utente viene reindirizzato alle scelte delle preferenze



6.2.2 2. Preferenze Dietetiche (Preferences)

Una schermata di configurazione critica per la raccomandazione. L'utente compila un form per specificare:

- *Regime Alimentare:* (es. Onnivoro, Vegetariano, Vegano).
- *Intolleranze e Allergie:* Selezionabili tramite tag (es. Lattosio, Glutine).
- *Tempo di preparazione:* Budget di tempo massimo desiderato per cucinare.

Questi vincoli vengono sincronizzati con il database Preferences su MySQL e fungono da filtri restrittivi durante l'invocazione delle API di Spoonacular.



6.2.3 3. Dashboard (Home)



Il centro nevralgico dell'applicazione. Fornisce un riepilogo a colpo d'occhio dello stato dell'utente. Mostra widget riassuntivi generati interrogando il database locale (Room), come ad esempio il numero di ingredienti attualmente presenti in dispensa, i pasti di oggi, i punti API rimasti di Spoonacular

6.2.4 4. Dispensa (Pantry)

Il core gestionale per l'inventario domestico. La schermata è divisa in tab logici (es. *In Uso*, *Da Comprare* *Non in uso*).

- **Inserimento ML Kit:** L'utente può aprire la fotocamera e inquadrare il codice a barre di un prodotto. Il sistema estrae il codice, interroga OpenFoodFacts per ottenere i dettagli (nome, foto, quantità) e pre-compila la scheda prodotto.
- **Inserimento Image Recognition:** Aggiungere elementi tramite ML
- In caso d'errore si chiede di farlo manualmente
- Tutti i dati vengono salvati immediatamente su SQLite per l'accesso offline e sincronizzati in background col server.



6.2.5 5. Pianificatore Settimanale (Piano)



L'interfaccia dedicata alla generazione e visualizzazione della dieta settimanale. Presenta un calendario interattivo. L'utente può richiedere la generazione di un nuovo piano: l'app contatta il backend, il quale incrocia le scorte presenti in Dispensa con le Preferenze Dietetiche e le ricette fornite da Spoonacular, restituendo un JSON strutturato con i pasti divisi per giorni (dal Lunedì alla Domenica) e per pasti (Colazione, Cena, Pranzo).

Ogni pasto può essere schiacciato e far apparire informazioni aggiuntive: calorie, proteine, grassi, carboidrati, ingredienti e i passaggi.

6.2.6 6. Ricette Custom

L'interfaccia dove si può visualizzare e creare nuove ricette custom.